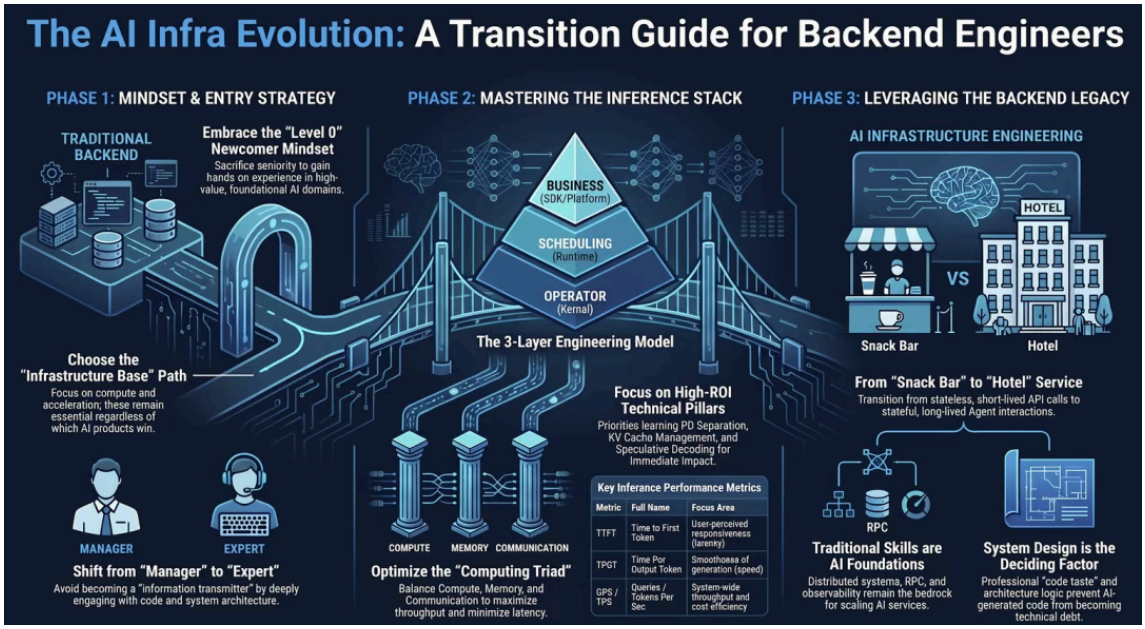
腾讯15年资深后台工程师，转战大模型推理的抉择

腾讯程序员 腾讯技术工程 2026年9月22日 03:55 广东 126人



作者：吴银光

> **导语：**本文讲述腾讯15年资深后台工程师，在 AI 时代如何入门大模型推理工程。全文围绕三条主线展开：从资深后台工程师到推理工程新人的抉择、推理工程的入门路径与自检方法、AI Infra 与通用 Infra 的同源与分野。



1. 前言

我工作了 15 年，2016 年加入腾讯，2022 年凭借搜索中台的工作晋升 T12，是一名资深搜索后台工程师。2025 年初，我切换到大模型推理工程团队，放下过往的资历与经验积累，以归零心态从头开始。本文既是我的自省回顾，也希望能给在 AI 时代犹豫迷茫的同行人一些鼓励，并与各位交流共勉，一起追赶浪潮、推动效率新时代的到来。

2. 大模型推理工程的零级新人

从 2024 年到 2025 年，我最大的变化是从资深搜索后台工程师和基干变成大模型推理工程的新人。"新人"不是一个谦词，是字面意义上的"新人"，如同实习生、应届生一般，没有高级头

衔，也不做任务分发。最初三个月，我的工作任务集中在性能评测、服务部署等大模型周边的琐碎事项；不过在这个过程中，我也抽时间深入阅读了 SGLang 源码。

为什么要从自己熟悉的领域离开，去当零级新人呢？这不是我一时兴起，也不是组织安排，而是经过反复考虑后作出的抉择。

2.1 AI 时代的选择

三十年前 PC 互联网来临时，我还是个孩子；十五年前移动互联网兴起时，我刚毕业只懂得选择高薪的 offer。今天，AI 的浪潮奔涌而至，而我将至不惑之年，正处于心智和经验的巅峰期。如果我此刻仍选择固守旧业，二十年后，子辈对我的追问，就如同今天我想问父辈的那句："四十年前改革开放迎面而来，当时你在做什么？"

2022 年底 ChatGPT 横空出世，让基于 Transformer 架构的大模型席卷全球。"更大的参数带来更高的智能基线"这一规律，至今依然有效。面对这又一次工业革命级别的机遇，我们能做什么？普通人又该如何抓住它？

大模型正在重塑每一个行业：2023 年 RAG 在搜索场景全面铺开，2024 年 AI 编程日趋成熟。全新的交互模式与技术栈正在孕育，因此，拥抱这波浪潮最好的方式，就是选择离大模型最近的方向，甚至直接切入大模型的核心领域。

如何切入？两三年前，尽管搜索产品在形态上相对贴近大模型应用，但其内部的召回、排序等细分领域依然大量沿用传统技术；而多轮对话、长上下文、Agent Runtime 等概念也尚未形成清晰的产品形态。我相信未来一定会诞生划时代的 Agent 产品，但在迷雾重重、看不清具体形态的当时，我选择进入大模型推理工程——因为无论上层产品如何演进，算力和推理加速这类基础设施，永远是不可或缺的底座。

2.2 管理者的技术困境

管理者绝不应该是信息传声筒，更不该是靠信息差生存的人。管理本身就是一门严谨的技术，需要持续学习、练习，投入大量精力。然而，面对一波全新的技术浪潮时，管理者很难做到 100% 专注地学习新技术，这往往会导致两难境地——要么技术学习流于表面，要么管理水平出现滑坡。

面对这种两难，传统路径给出的答案往往是放弃技术：许多技术管理者最终转型为纯粹的"职业经理人"。他们的核心职责应当是构建务实高效的团队文化，激发工程师创造更大价值。但现实中，我们经常看到一些管理者既想享受管理的红利，又不愿放弃技术的头衔，从而产生不少怪现象：在学术界，有人名下挂名大量论文，或在学术会议上侃侃而谈，到了 Q&A 环节却只能坦言"具体细节是学生做的"；在工业界，也有负责人名下挂着诸多大项目，虽无法深度参与，却能借此争取个人晋升。值得庆幸的是，如今这类现象已在减少，越来越多的评估开始回归本质——技术与职级终究要服务于项目贡献，为组织创造最大价值的人，才应该获得相应的回报。

当 AI 时代轰然而至时，作为一个依然怀有技术信仰的程序员，我并不想过早地蜕变为职业经理人；可要真正扎进大模型技术，又必须投入海量时间与极致的专注。组织不可能允许一个人带着高阶管理的头衔，去当一名大模型方向的"初学者"。因此，当面对需要全力倾注的重大技术变革时，我选择卸下管理的头衔与包袱，以纯粹的心态重新出发。

今天回看这段抉择，对于想切入新领域的管理者或资深工程师而言，除了我这种归零重来的方式，其实还有第二条路径：AI Infra 与通用 Infra 本就非常接近。尤其是在 Agent 产品路线日益清晰的今天，从通用 Infra 逐步渗透到 AI Infra，是一条顺畅的自然演进路线。在一个 AI 产品团队中，AI Infra 绝非仅仅等同于 GPU 计算加速，它背后依然需要极度扎实的平台化建设、研效提升与稳定性保障等传统基础设施支撑。从这些熟悉的切入点入手，思考如何对通用 Infra 进行"AI 化改造"，同样可行。

2.3 组织变化的契机

除了 AI 时代的呼唤与技术管理者的困境，组织架构的变动，也成为了我做出决定的催化剂。

2024 年，我随大团队合并至新部门。在此后的几个月里，小组一度缺乏主线任务，我不得不带着大家到处"找活"，处理各种边缘杂事，维持小组稳定性变得愈发艰难。随后，组织有意将我们团队并入新部门的产品主链路，同时需要我去掉管理头衔。在早期的试探性沟通中，我坦然接受了这个调整——这契合当时部门提倡的"以事为先"理念，也与我一直信奉的"工程师文化"不谋而合：**决定一个人价值的，从来不是他的头衔，而是他正在做的事情。** 于是，我选择从自己熟悉的技术方向切入，着手熟悉产品主链路的研发体系与业务形态，并通过一次耗时三个月的独立代码重构，摸清了核心链路的全流程。

在这个过程中，内心并非没有过摇摆。从 A 部门到 B 部门，一方面研发体系完全不同，另一方面也面临身份切换与角色落差，我也曾动过"是不是再找个管理的坑位"的念头，甚至也确实接触过这样的机会。但每一次面临抉择，我都用同一句话提醒自己：**在 AI 时代，你应该留在离 AI 最近的团队。** 当时在我看来，搜索就是离 AI 最近的产品形态——Transformer 架构诞生于搜索巨头，RAG 是传统搜索的自然延续，而对话框里的大部分需求，本质上依然是对信息检索与处理的诉求。比起管理头衔，我更看重技术方向的长期价值。

到了 2025 年初，在我对新部门的基础设施有了深刻理解后，正好看到了部门内 AI Infra 团队的招人信息，经过与多位团队负责人的深度沟通，我最终加入该团队，投身大模型推理工程。

3. 大模型推理工程入门指引

大模型推理工程是一项涉及模型结构、GPU 底层硬件特性、分布式系统与调度的复杂系统工程。它不是单纯的深度学习模型部署，核心矛盾是算力、显存、通信三者之间的权衡：Prefill 阶段是计算密集型，Decode 阶段是显存带宽密集型；KV Cache、Continuous Batch、Prefix Cache、TP 等技术，本质上都是为了在硬件约束下，以最低成本换取更高的吞吐与更低的时延。

知识点覆盖面较广，建议采用先总后分的思路：先建立体系化认知，再逐个击破。

3.1 对模型结构建立体系化认知

当前主流大模型均采用 Transformer 架构，入门第一步便是对模型结构建立体系化认知。

- 熟悉 Transformer 模型结构，知道它的核心组件，如 Embedding、Attention、FFN、Normalize、Residual、LM Head，能画出模型结构简图。

- 了解深度学习技术体系的基本原理和关键技术，如感知机、多层神经网络、梯度下降、误差反向传播、超参数、CNN、RNN、Seq2Seq、Attention 结构等。
- 了解深度学习发展史上关键人物、关键公司、关键投资的故事。

可以借助 AI 学习，但通常不够系统，建议阅读书籍或者系统概述类文章。推荐书籍：

- 深度学习和自然语言处理基础书籍：《深度学习入门：基于 Python 的理论与实现》、《深度学习进阶：自然语言处理》
- 深度学习人文故事：《深度学习革命》

相对来说，Transformer 模型结构反而是上述三类信息里最简单的，推荐借助 AI 或者在知乎上搜索相关文章学习（尽管知乎 APP 的开屏广告很烦人，但它仍是国内大模型知识和讨论密度最高的平台）。

自然语言处理如何演进到 Transformer 模型结构，我之前做过一篇总结：《从词向量到大模型：NLP 技术演进浅记》。

3.2 对大模型推理工程建立体系化认知

在分析技术问题时，无论是单个模块还是完整领域，首要工作是明确两个问题：要解决什么问题，有什么评估体系。掌握了核心目标和评估体系，对系统中每个组件就会有更本质的判断：该组件如何支撑整体目标，以及如何量化它的优劣。

大模型推理工程的顶层目标与传统软件工程一致，包含功能、性能、成本、稳定性等维度。而评估体系包含以下核心指标：TTFT、TPOT、QPS、TPS、Concurrency（并发数），还有成本。围绕这些指标，在系统层面做的每一项优化，都可以归纳为算力、显存、通信三者之间的资源置换。据此，我们可以抽象出三层模型：顶层是业务目标，中间层是量化评估标准，底层是具体实现手段。

落到工程体系与组织分工上，上述"实现手段"可进一步划分为三个子方向：

- **业务层**：负责推理系统与上层业务的对接。包含推理服务 SDK、推理平台、服务部署与监控、面向业务定制的请求流量调度等模块。
- **调度层**：作为模型推理运行时载体，直接决定系统核心性能，绝大多数推理加速优化都落地于此层。典型技术包括分布式并行、Prefix Cache、Overlap Scheduler、Continuous Batch、Chunked Prefill、Speculative Decoding 等。
- **算子层**：负责高性能基础计算单元的实现，聚焦单点计算性能优化，例如 QKV 计算、Normalize、Linear 计算。

分层架构的核心价值，是降低大规模系统与大型研发团队的复杂度，让各方向的团队聚焦自身职责边界。而对于业务规模较小、系统复杂度偏低的场景，则不必做如此精细的切分，研发人员可以跨多个层次开展工作。

同样可以借助 AI 学习，建立一些基础概念。对于有开发经验的程序员来说，形成体系化的认知是比较简单的，难的是深入细节，譬如 Prefix Cache、Overlap Scheduler 是怎么实现的。

这方面建议通过源码学习，nano-vLLM、mini-sglang 都是不错的入门代码，我之前做过阅读总结：

- 基于 nano-vLLM 学习大模型推理关键功能
- 基于 mini-sglang 学习大模型推理关键功能

3.3 了解主流模型结构的演进趋势

在大模型时代，传统的“算法”与“工程”岗位边界日益模糊。对于推理工程而言，只懂系统架构已远远不够，深入理解模型结构同样是必修课——因为新一代模型结构的演进，本质上是算法创新与工程优化深度结合的结果。从最初的 Full Attention + Dense FFN，发展到如今的稀疏注意力、线性注意力、混合架构、MoE，以及 Pre-Norm、RoPE、MLA、KV Cache 跨层复用等技术，仔细梳理这条演变路径不难发现：模型在变得更智能的同时，也在朝着大幅降低计算复杂度、更适配长上下文的方向演进，以更低的推理成本实现更高的吞吐。DeepSeek 就是这方面的典型代表：注意力层的 MLA、HCA、CSA 等设计，以及 DSpark 推测解码，都是算法创新与工程优化结合的案例。

进一步思索，会发现现阶段大模型的结构演进，本质上是一门“实验科学”。**很多创新都是可想到的，当创新点子变得廉价，昂贵的就是验证思路的实验是否可以更好更快的完成**。这时候 infra 显然极其重要，理解模型结构，搞明白演进背后的动机，可以帮助我们理清趋势，进而做出更完备、更有前瞻性的 infra。

推荐阅读：

- 《The Big LLM Architecture Comparison》
- 《从 305 GB 到 7.4 GB：大模型 KVCache 架构演进全景》

3.4 深入某个优化方向

大模型推理工程涉及的知识体系极其庞杂。在建立起宏观的体系化认知之后，建议结合具体的业务痛点或个人兴趣，选定一个细分的主题进行分析学习。目前来看，投入产出比较高且业界高度关注的子方向包括：

- **Prefill / Decode（PD）分离**：通过将 Prefill 与 Decode 阶段解耦部署，能显著提升整机吞吐与资源利用率，对于算力预算有限、卡资源紧缺的团队而言尤为关键。尽管开源已经有比较好的实现，但是结合自身的业务场景，怎么适配到性能最佳，也很有挑战。
- **分布式 KV Cache**：针对具备多轮交互、长上下文特点的业务场景，能够大幅提升 Prefix Cache 的命中率，显著降低重复计算成本。
- **推测解码（Speculative Decoding）**：作为算法创新与工程优化深度结合的典范（例如 DSpark 等架构），它巧妙利用了 Decode 阶段算力未饱和的特点，大幅度提升系统吞吐。

除了这些大方向，还有很多有趣且独立的"小技术点"值得深挖。例如我此前曾对 Overlap Scheduler 做过系统分析《大模型推理加速：Overlap Scheduling 的深入剖析与性能权衡艺术》。当顺着底层执行链路深挖下去时，往往能发现一些被常人忽略的暗礁，譬如在特定约束下，Overlap Scheduler 反而可能会破坏 Continuous Batch 的批处理效率。这种对微观机制的钻研可以通过 Demo 代码完成验证，学习沉淀也会更加扎实。

4. 大模型推理工程入门自检练习

在 AI 广泛使用的今天，如果学习知识仅仅停留在"被动输入"而缺乏"主动输出"，极易陷入"看时全都会，做时全不会"的虚假掌握感。为了帮助大家检验对推理底层机制的真实理解，我结合自己近一年来的实战项目经验，设计了以下三个练习点。

4.1 同批次共享前缀的注意力优化

提到前缀共享，大家第一反应往往是跨请求/跨批次的 Prefix Cache。但如果是在同一个 Batch 内存在多个带有公共前缀的请求——例如：`请求1 = [a, b, c, 1, 2, 3]`，`请求2 = [a, b, c, 4, 5, 6]`，我们该如何避免针对前缀 `[a, b, c]` 的重复 Prefill 计算与 KV Cache 重复写入？

这类需求在个性化推荐场景中极为常见。例如在文档打分任务中，输入形式通常为 `user_info + doc_N`：前缀 `user_info` 代表用户属性（在同一个批次中完全相同），后缀则是不同的待评估文档。

将该问题抽象出来，本质就是：同一 Batch 内的共享前缀的注意力计算优化。要完整解答并实现这一优化，需要融会贯通以下多维度的底层知识：

- 深刻理解因果掩码（Causal Mask）的构造；
- 在代码层面熟练掌握 KV Cache 的申请和使用；
- 理解 FlashAttention / FlashInfer 等主流 Attention Kernel 的接口设计；
- 了解 GPU Thread Block 的并行特点。

可以先看看自己是否能独立想到一些解决方案，然后再看我做的系统分析：《大模型推理加速：共享前缀优化技术的全面解析》。在实际业务落地中，同批次共享前缀往往只是单点问题，还需要与传统后台技术结合落地，这些周边结合点（如个性化的模型结构、业务系统的嵌入等）往往更有难度。

4.2 Beam Search 的工程实现

Beam Search 技术最近一两年在搜广推领域较为火热，这与快手发布的 OneRec 论文（**OneRec Technical Report**）密切相关，不少公司也已经将这项技术落地到内部的搜索、推荐项目中。

Beam Search 的技术原理较为简单直接：每一步在 backbone 和 LM Head 执行完后，挑选 token 时不再只保留最优的一个，而是保留多个候选 token，作为多条子路径继续推导；每一步选择下一个 token 时，将多条子路径的候选汇总到一起，按整条子路径累积的对数概率（logprob）排序，保留得分最高的 Beam Width 条继续推导，如此迭代，直到生成结束后选出最佳子路径。

这里涉及到几个关键的技术点：

- KV Cache 复用。多条子路径有公共的前缀，理应共享前缀，但它们是同一个请求扩散出来的多个子请求，需要理解推理引擎的 KV Cache 实现，然后做到高效的复用；

- 高效率的剪枝。在选择下一批合法 token 的时候，需要将所有路径的候选合并后统一计算、统一筛选。当 Beam Width 非常大，譬如 BW=1024，需要从 1024 * Vocabulary Size 个候选里挑选出最佳的 1024 个，计算量极大；
- 约束解码。Beam Search 应用于推荐场景时，通常我们会限制只能生成合法的 DocID，因此每一步生成都要有词表约束，怎么才能做到最高性能；
- 和推理框架的结合。相对于 Greedy（贪心解码）场景，Beam Search 是比较小众的功能，怎么避免小众功能成为主流功能迭代的负担。

可以在 mini-sglang 等小型引擎上尝试实现。我去年在 SGLang 上实现高性能的 Beam Search，当时做到业界 SOTA 性能（现在有更强的定制引擎实现了更好的性能），也写了一篇总结分享《大模型推理引擎中的 Beam Search：工程挑战、主流实现与 SGLang 深度优化》。SGLang 官方开发者也在我开发的分支基础上做重构融合，最终合并到 SGLang v0.5.19。

4.3 读懂大模型方向的技术文章

大模型方向的技术文章非常多，包括论文、论文解读、模型结构分析等，入门的重要标志是能看懂这些文章。阅读 HCA/CSA、混合架构、DSpark 等技术论文或者文章时，可以理解基本原理。

5. AI Infra：传统 Infra 版图的自然扩张

在掌握了常规大模型推理加速方法之后，若切换到产品视角审视大模型推理工程，会发现它并非一个从零诞生的全新领域，而是传统 Infra 版图的自然扩张。二者同源：AI Infra 的多数技能点都建立在通用 Infra 的基础之上；二者也有分野：AI Infra、Agent Infra 在资源模型与服务范式上，已显著区别于传统后台系统。对一名 Infra 工程师而言，学习 AI Infra 不是推倒重来，而是知识边界的自然拓展——传统后台开发沉淀的系统能力，在 AI 时代依然极具价值。

5.1 传统 Infra 知识体系依然是大模型推理服务底座

今天我们对外提供大模型推理 API 服务，其底层依然搭建在传统 Infra 能力之上：微服务、分布式、RPC、存储、可观测体系、研效体系都不可或缺。

一套高性能、能力强大的大模型推理接口，是模型产品的核心组件。但如果团队开发者只掌握 vLLM/SGLang 这类推理引擎，这套系统只能停留在 demo 阶段。当用户规模增长，它必然会遇到大规模分布式系统的经典难题，这些问题高度依赖传统 Infra 能力：

- 流量预估、容量规划、机房与集群资源规划；
- 风险隔离、稳定性保障，同时保障迭代效率，搭建 CI/CD 体系；
- 服务发布、容器调度、全链路可观测体系建设；
- 微服务 / 单体架构取舍，有状态、无状态服务边界划分。

5.2 AI Infra：资源约束迥异，但系统思想大量继承自传统 Infra

大模型推理工程虽属于新兴领域，诞生了 Continuous Batch、Overlap Scheduler 这类专属技能点，但其底层大量工程思想，都能在传统系统工程里找到源头。以 PagedAttention 为例，

本质是把操作系统虚拟内存页表的思想搬到了显存管理上——逻辑上连续的 KV Cache 被切成固定大小的页块，通过块表（Block Table）映射到物理上离散的显存页。类似的思想迁移还有很多：TP、CP 等模型分布式并行方案，其核心目标与传统分布式系统一致：拆分计算、分摊负载；CUDA Stream 多流机制，思想上对标 CPU 多线程；分布式 KV Cache 的分片、生命周期管理方案，也可以大量借鉴分布式存储系统的成熟经验。总而言之，GPU 相关工程并非一套完全割裂的知识体系。在 CPU 侧深耕多年的资深后台工程师，完全可以把沉淀的系统思维迁移到 GPU 推理工程中。

5.3 传统后台开发的价值重估与 Agent Infra 新战场

5.3.1 传统工程经验在 AI 时代更显重要

当下 AI Infra 领域的招聘方普遍偏好有训推框架、内核算子开发等经验的 AI Native 人才，但我认为传统后台工程师的转型价值被低估。AI 专项人才虽可以更快上手，但传统工程师凭借扎实的系统设计功底，依托 AI 工具辅助，也可以很快追上，并发挥自身的经验优势。

AI 普及替代了基础编码等重复工作，让代码评审、架构设计成为核心事项。传统工程师长期积累的代码品味、架构思维与复杂系统把控能力，在 AI 时代愈发重要，可以有效规避 AI 生成代码的架构混乱、冗余与隐性风险。

由此我判断，在当前 AI Infra 人才稀缺的情形下，让有热情的后台开发工程师扩展知识边界，进入到 AI Infra 领域，在团队管理和人才培养上都是不错的选择。

5.3.2 技术方向的分层需求：越往底层需求越少、越往上层需求越多

AI Infra 行业具备清晰的分层特征：越底层共性越强、需求越少，越上层业务差异化越大、人力需求越旺盛，整体呈现"底层标准化、上层差异化"的格局。以大模型推理工程为例，各层级需求特征如下：

- **算子层**：模型结构、硬件生态将趋于收敛，新算子、新结构适配类似 Linux 内核迭代，核心价值极高、全行业可复用，但无需大量人力重复研发，仅需少量头部团队持续迭代适配新模型、新硬件即可，整体人力需求极少。
- **推理引擎调度层**：通用调度能力已被开源生态成熟覆盖，绝大多数业务无需深度定制，仅少数业务绑定的场景存在少量定制需求，整体人力需求有限。
- **业务层**：自身业务场景的定制适配、链路优化、稳定性打磨等差异化工作，是人力需求比较大的地方。

业务层的差异化工作，正是传统后台工程师迁移系统设计功底、发挥经验优势的主战场。

5.3.3 Agent Infra：传统后台开发者的新战场

随着 AI 底层底座、算子、推理引擎逐步标准化、开源化，底层差异化竞争空间持续收窄，行业创新重心全面上移，Agent Infra 成为传统后台开发者最适配的新战场。

Agent 是通用智能与垂直业务的衔接点，不同场景的 Agent 使用形态、运行逻辑、工程诉求差异极大：例如科研场景，OpenAI 可调度万级 Agent 求解 NS 方程，而普通场景仅需单

Agent 完成表格整理等轻量化工作，但海量个人场景会形成更大的量级规模。场景的高度差异化，让 Agent 产品与 Agent Infra 拥有巨大的工程落地与优化空间。

从服务范式来看，Agent 时代彻底重构了传统的后台服务模式。传统 API 服务是"小卖部模式"：短请求、无状态、耗时极低、即调即走、资源即时释放。而 Agent 服务是"旅馆模式"：以多轮交互、长耗时、有状态、长生命周期为核心特征，需要依托记忆存储、工具调用、外部资源交互完成任务，会话持续驻留、状态频繁变更。

该范式变革带来了全新的工程挑战：长会话调度、状态持久化、长短记忆管理、工具沙箱隔离、Agent 可观测与效果评估等一系列全新基础设施问题，而这类问题均属于传统分布式、系统工程的核心范畴，是传统后台经验可以发挥的地方。

6. 结语

本文的知识经验与行业分析，均来自个人经历与观察，仅代表个人观点。作为工作了 15 年的“互联网老人”，**在这波 AI 浪潮中，既有对新时代变革的兴奋，也有对国内互联网行业年龄歧视的错愕，而时光就在徘徊犹豫中溜走，不做选择就是最坏的选择。**尽管我觉得自己还能再干十年、二十年，直到脑子不再转动，但时代滚滚向前，过去的经验可能发扬光大，也可能被历史洪流冲刷掉。技术革命带来的每一次重置，也不过是让自己回到 15 年前的起跑线上，相信自己，继续跑下去。



腾讯技术工程

腾讯技术官方号。腾讯技术创新、前沿领域发布解读平台。

685篇原创内容

公众号

阅读 1.7万